[Flutter] 예외처리(Exception Handling)

✅ 용어 정리

용어	뜻
Exception (예외)	프로그램 실행 중 발생하는 *비정상적인 상황* (예: 0으로 나누기, 파일 없음, 서버 응답 없음 등)
Exception Handling (예외처리)	예외가 발생했을 때, *앱이 죽지 않도록 안전하게 처리하는 코드 구조*
try-catch-finally	예외 처리 구문의 대표적인 구조
에러(error) vs 예외(exception)	Dart에서는 대부분의 **에러**는 복구 불가능 (OutOfMemoryError), **예외**는 복구 가능

✅ 1. 왜 try-catch가 필요한가?

❌ 에러가 처리되지 않으면 앱이 강제 종료됨

```
void main() {
  int result = 10 ~/ 0; // 에러 발생: IntegerDivisionByZeroException
  print(result);
}
```

> 이렇게 되면 앱이 크래시돼요. → 사용자는 아무런 안내 없이 튕기게 됨.

✅ 2. 기본 try-catch-finally 구조

```
try {
  // 예외가 발생할 수 있는 코드
} catch (e) {
  // 에러가 발생했을 때 실행
} finally {
  // 항상 실행되는 코드 (에러 여부와 무관)
}
```

✅ 3. 실제 예제: 서버 통신

```
Future<void> fetchData() async {
  try {
    print("서버에 요청 중...");
    await Future.delayed(Duration(seconds: 2)); // 가상의 네트워크 지연
    throw Exception("서버 오류"); // 일부러 에러 발생
  } catch (e) {
    print("에러 발생: $e");
  } finally {
    print("로딩 상태 해제 (에러와 무관하게 실행)");
  }
}
```

✅ 4. `catch`에 들어오는 값은?

타입	설명
`e`	예외 객체 (에러 정보)
`stackTrace`	에러가 발생한 위치(파일/라인)를 포함한 정보 (디버깅용)

```
catch (e, stack) {
  print("에러: $e");
  print("스택: $stack");
}
```

✅ 5. 특정 예외만 골라서 처리할 수 있어요: `on`

```
try {
  int result = 10 ~/ 0;
} on IntegerDivisionByZeroException {
  print("0으로 나눌 수 없습니다!");
} catch (e) {
  print("기타 에러: $e");
}
```

✅ 6. `rethrow`로 예외를 다시 던질 수 있어요

에러를 잠깐 잡았다가 다시 위로 전달하고 싶을 때 사용합니다.

```
void riskyFunction() {
  try {
    throw Exception("문제 발생!");
  } catch (e) {
    print("로그 남기기: $e");
    rethrow; // 다시 에러 던짐
  }
}
```

```
void main() {
  try {
    riskyFunction();
  } catch (e) {
    print("최종 처리: $e");
  }
}
```

✅ 7. finally는 언제 써야 할까?

리소스 정리, 상태 초기화, 로딩 중지, 연결 해제 등 **항상 실행되어야 하는 정리 작업**에 사용합니다.

```
try {
  // 서버 요청
} catch (e) {
  // 에러 처리
} finally {
  // 로딩 상태 끄기
}
```

✅ 8. async 함수에서 try-catch 사용 시 주의할 점

```
Future<void> loadData() async {
  try {
    await Future.delayed(Duration(seconds: 1));
    throw Exception("에러 발생!");
  } catch (e) {
    print("에러 잡힘: $e");
  }
}
```

- **await가 있는 경우** 반드시 `async` 함수 내에서 `try-catch`로 감싸야 동작합니다.

- `await` 없이 그냥 `Future`만 썼다면 `catch`에 잡히지 않을 수도 있어요.

✅ 9. Flutter 앱에서 실전 사용 예

🌀 로딩 UI 처리

```
bool _isLoading = false;

void loadData() async {
  setState(() {
    _isLoading = true;
  });

  try {
    await fetchData();
  } catch (e) {
    showDialog(context: context, builder: (_) => AlertDialog(
      title: Text("에러 발생"),
      content: Text("$e"),
    ));
  } finally {
    setState(() {
      _isLoading = false;
    });
  }
}
```

✅ 10. 정리 – try-catch-finally의 목적

목적	설명
에러 방지	앱이 죽지 않게 함
사용자 경험	에러 메시지를 유저에게 보여줌
디버깅	스택 추적/로그 남기기

목적	설명
정리 작업	무조건 실행할 정리 코드 작성

✅ 시각적 흐름 정리

```
[시도 (try)]
   ↓
[에러 발생 ❌] → 정상 실행 계속
[에러 발생 ✅] → [catch로 이동]
   ↓
[finally] → 무조건 실행
```

📌 보너스 요약 카드

구문	설명
`try`	예외 발생 가능성 있는 코드
`catch (e)`	예외를 처리함
`catch (e, stack)`	예외 + 스택 트레이스
`on 에러타입`	특정 예외만 처리
`finally`	항상 실행됨
`rethrow`	예외 다시 던짐